

A5 — Integration Evidence · Team 16 (Private Wellbeing Intake & Booking)

Team: 16 — Wellbeing · **Pod:** 4 — Student Support **Deployed base URL:** `https://wellbeing-intake.vercel.app/api/v1` (Vercel, region `sin1`) · database: Supabase project `jtwrgfcmfghfklddyatv` (ap-southeast-1) **Commit / deploy ID evidenced:** `818596e` for §5a and §6a, `73b18dc` for §6b, `27d4920` for §3, §4 and §5b, `f2b620c` for §2 (each reported by `GET /health` as `"version"` at capture time) **Evidence captured on:** 2026-09-21, 05:33–07:56 UTC+7 (§3–§6) and 18:19–19:11 UTC+7 (§2). Timestamps below are ISO-8601 **UTC** (z), exactly as captured; add 7 h for local time. **Captured by:** Teerapat Sukkasem (6731503015), Team 16

Scope of this document. Team 16 is paired with **one** partner: **Team 14 — Helpdesk**. Team 14 provides the Provider proof (§2). We are not paired with Team 01 (Identity) or Team 20 (Notification Hub), so:

- the webhook proofs (§3, §4, §6a) are real captures from the deployed system, **run by Team 16 itself** against our built-in contract mock (`/api/v1/mock/hub`), which verifies the real HMAC signature, platform envelope and `data` allowlist over real HTTP. They prove our side of the contract; they are not a partner's confirmation, and we do not present them as one;
- the Consumer proof (§1) is out of scope — see §1.

§3, §4 and §5b were first captured at 05:33–05:50 UTC+7 and **re-run at 07:56** with identical results, because Vercel's Hobby plan keeps server logs for about one hour and the first run's log lines had expired before they could be screenshotted. Both runs are kept in the raw captures.

Raw captures: [docs/evidence/](#).

Every item below must be a real capture from the deployed system — nothing reconstructed, nothing mocked up after the fact. Anything still marked `TODO` is incomplete.

Redaction rules (apply to every screenshot and log)

- ☒ Seeded demo users only — no real student record anywhere (platform PRD §6, BR-18).
- ☒ Secrets masked: signing secret, machine credential, service-role key, LLM key, bearer tokens (show first 6 characters at most).
- ☒ No request content (description, free text, preferred times) and no triage level in any capture (NFR-07). Use metadata-only views. *One deliberate exception: §5a shows the request it replays, as this template asks — its description is the obviously fake sentinel `demo-sentinel-A5`, with the routine level.*
- ☒ Each capture shows its correlation ID (`X-Correlation-Id`) so the log lines and screenshots can be tied together.

Summary

#	Proof	Partner team	Our PRD reference	Status
1	Consumer	— (not paired with Team 01)	BR-26, FR-03	Out of scope — identity runs in fixture mode
2	Provider	Team 14 — Helpdesk	§11.1 GET /services, NFR-18	Done — our server log and Team 14's Postman capture share <code>team14-a5-0001</code>
3	Webhook receiver	— (self-run)	FR-24 (PRD revision 3), POST /webhooks/notification-hub	Done (self-run)
4	Webhook sender	— (contract mock hub)	FR-18, NFR-17, §11.5	Done (mock hub)
5	Idempotency	— (ours)	K-17 <code>submission_key</code> ; <code>eventId</code>	Done — 5a and 5b
6	Degradation	— (contract mock hub / LLM)	K-18/K-19, FR-23, BR-24	Done — 6a (hub outage) and 6b (LLM rate-limited), both with automatic recovery

1. Consumer Proof — we call a partner's API

Status: out of scope for this submission. Our designed consumer integration is Team 01 — Student Identity: `src/lib/identity/adapters.ts` verifies a Team 01 access token against their published key set when `IDENTITY_MODE=campus` (BR-26). Team 16 is **not paired with Team 01**, so no real Team 01 token exists to capture, and we will not fabricate one.

What is true of the deployed system instead:

Field	Value
Identity mode deployed	<code>IDENTITY_MODE=fixture</code> — seeded demo users via Supabase Auth, which BR-26 permits when Team 01 is unavailable
Adapter code	<code>src/lib/identity/adapters.ts</code> — the only module that verifies a credential. Switching to <code>campus</code> is configuration (<code>IDENTITY_JWKS_URL</code> , <code>IDENTITY_ISSUER</code>), not a code change (NFR-14)
Honest limitation	The <code>campus</code> path has never been exercised against a real Team 01 token, and it is not covered by our automated tests. Only fixture mode is proven.

Our only partner, Team 14 — Helpdesk, exposes no API that this product needs to consume, so we did not invent a call to one for the sake of this section.

2. Provider Proof — a partner calls our API

What this proves: Team 14 — Helpdesk successfully consumes a Team 16 endpoint, both by hand (Postman) and from their own deployed backend.

Endpoint: `GET /api/v1/services` — public, returns only the seeded service catalogue, so a partner can call it with no privacy exposure. The agreed contract is [integration/team14/Team14-Integration-Contract.md](#) (v1.2):

Helpdesk uses the catalogue to suggest a Wellbeing service on a ticket and stores only the stable `slug`. No student data crosses the boundary in either direction, and no ticket text is sent to us.

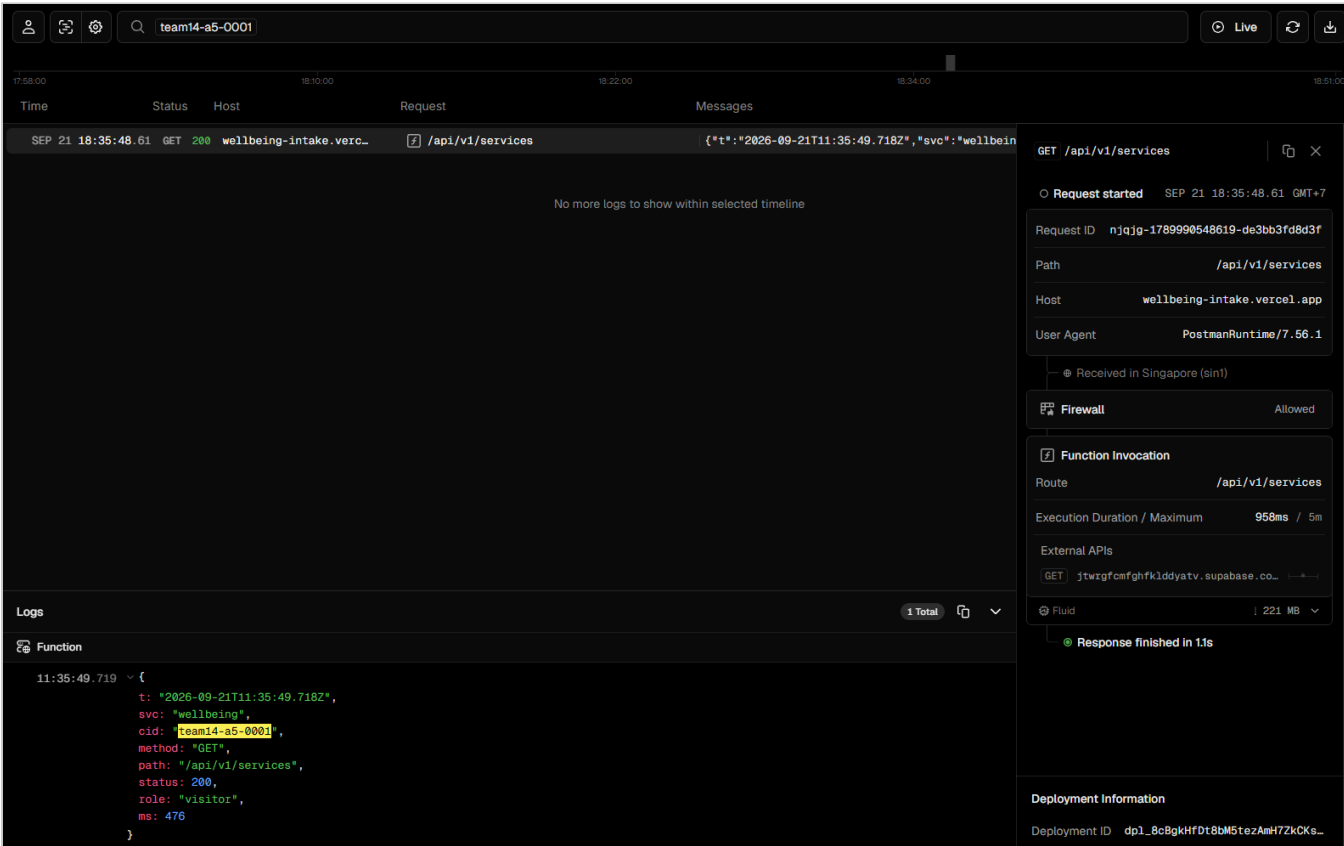
2a. The evidenced call — correlation ID `team14-a5-0001`

Field	Value
Our endpoint URL	<code>https://wellbeing-intake.vercel.app/api/v1/services</code>
Calling partner	Team 14 — Helpdesk · contact: Pupattarak Masomjit (6731503115)
Partner's client	Postman (User-Agent: PostmanRuntime/7.56.1)
Request timestamp	2026-09-21T11:35:49.718Z — 18:35:48 UTC+7 (server time, from our log)
Correlation ID	<code>team14-a5-0001</code> — sent by Team 14, echoed in our response, written to our log
HTTP status returned	200
Deploy evidenced	<code>f2b620c</code> , Vercel production, received in Singapore (<code>sin1</code>)

Internal request log (Vercel function log line — method, path, status, correlation ID, caller role; no body):

```
{ "t": "2026-09-21T11:35:49.718Z", "svc": "wellbeing", "cid": "team14-a5-0001", "method": "GET", "path": "/api/v1/services", "status": 200, "role": "visitor", "ms": 476 }
```

`role: "visitor"` is correct: the endpoint is public, so Team 14 presents no credential and receives exactly what any signed-out visitor receives.



The log search for `team14-a5-0001` returns exactly one request. Team 14's optional label `team14-a5-0002` (health check) was searched for and returned no request: it was not sent, so nothing is claimed for it. Their recovery call `team14-a5-0003` arrived later and is recorded in §2c.

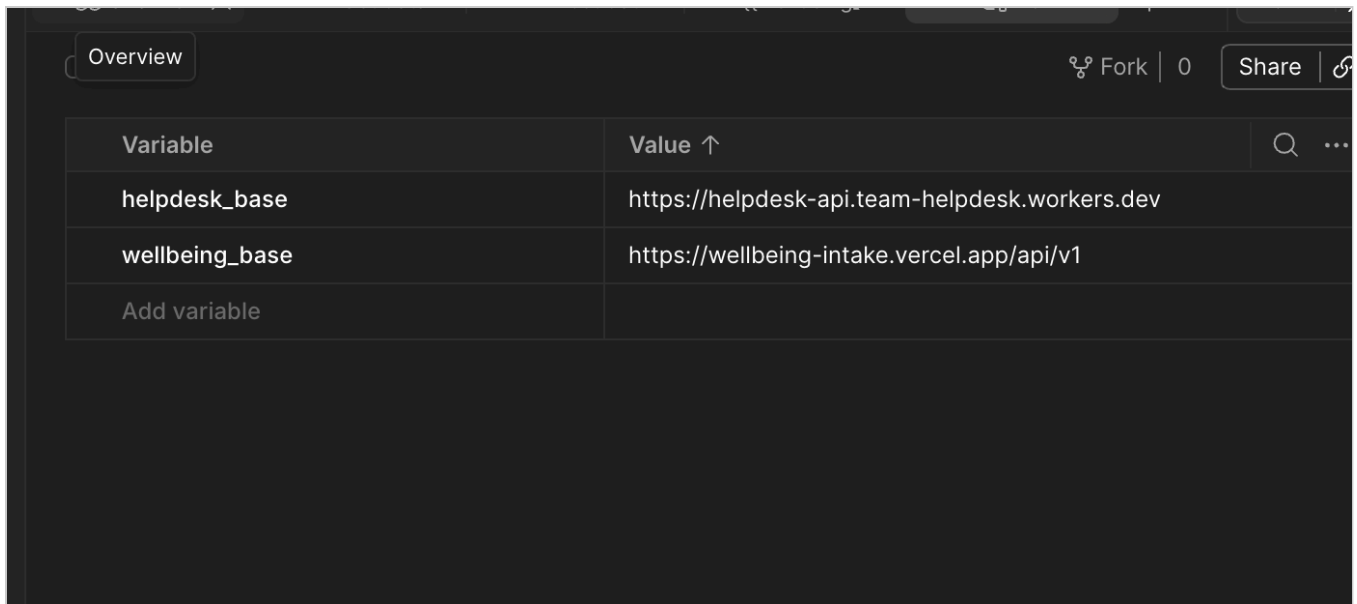
Partner confirmation — Team 14's own Postman capture of the same request, supplied by Team 14:

What their screenshot shows	Matches our log
GET {{wellbeing_base}}/services, with wellbeing_base = https://wellbeing-intake.vercel.app/api/v1 (their Postman environment, second image)	path /api/v1/services, method GET
Request header X-Correlation-Id: team14-a5-0001	cid: "team14-a5-0001"
200 OK, 1.4 KB, in 1.44 s	status: 200; User-Agent: PostmanRuntime/7.56.1
Body: our catalogue, starting with counselling (id 689dfeea-...) then health-clinic	the seeded services, ordered by name

Their capture shows the request header they sent rather than the response headers; the time of the call is taken from our server log (2026-09-21T11:35:49.718Z), where the search for that label returns this one request only.

The screenshot shows a web browser's developer tools interface. At the top, the URL bar displays `{{wellbeing_base}}/services`. Below the address bar, the 'Headers' tab is selected, showing a table with one header: `X-Correlation-Id` with the value `team14-a5-0001`. The 'Body' tab is also visible, showing a JSON response. The response status is `200 OK` with a response time of `1.44 s` and a size of `1.4 KB`. The JSON body is as follows:

```
{
  "services": [
    {
      "id": "689dfeea-2a43-4aa9-87b4-55cfacaf29bc",
      "slug": "counselling",
      "name": "Counselling",
      "whatFor": "Talking through stress, low mood, anxiety, relationships, grief, or anything that is weighing on you. You do not need a diagnosis or a 'big enough' problem.",
      "firstSession": "A relaxed 50-minute conversation with a counsellor about what is going on and what might help. Nothing is decided for you.",
      "whoWillKnow": "Only the counsellors of this service. Not your lecturers, not your parents, not other students."
    },
    {
      "id": "e82330e2-747f-42ce-831c-a0fe844036f3",
      "slug": "health-clinic",
      "name": "Health clinic",
      "whatFor": "Everyday physical health: vaccinations and flu shots, feeling unwell, minor injuries, medical certificates, and general health questions."
    }
  ]
}
```



The screenshot shows a Vercel environment variables configuration page. At the top, there is a tab labeled 'Overview' and buttons for 'Fork' (with a count of 0) and 'Share'. Below this is a table with two columns: 'Variable' and 'Value'. The first row shows 'helpdesk_base' with the value 'https://helpdesk-api.team-helpdesk.workers.dev'. The second row shows 'wellbeing_base' with the value 'https://wellbeing-intake.vercel.app/api/v1'. At the bottom of the table is a button labeled 'Add variable'.

Variable	Value ↑
helpdesk_base	https://helpdesk-api.team-helpdesk.workers.dev
wellbeing_base	https://wellbeing-intake.vercel.app/api/v1
Add variable	

2b. Their deployed backend calling us — not just a manual test

Team 14's Helpdesk API (<https://helpdesk-api.team-helpdesk.workers.dev>, Cloudflare Workers) proxies our catalogue at `GET /wellbeing/services`. Team 14's evidence plan states that this proxy generates its own random correlation ID for every call; the log line below carries an ID of exactly that kind (`team14-a5-<uuid>`), which is how we attribute it to their backend rather than to a person:

```
{ "t": "2026-09-21T11:19:24.677Z", "svc": "wellbeing", "cid": "team14-a5-c006d636-c04e-4d40-9653-2c1138a03ad4", "method": "GET", "path": "/api/v1/services", "status": 200, "role": "visitor", "ms": 371 }
```

This request shows no browser or Postman user agent: it is a server-to-server call from their Worker, which is how the contract asks them to call us (we send no CORS headers).

The screenshot displays a log management interface with a table of requests and a detailed view of a specific request on the right.

Time	Status	Host	Request	Messages
SEP 21 18:35:48.61	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:35:49.718Z", "svc": "wellbein
SEP 21 18:19:23.39	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:19:24.677Z", "svc": "wellbein
SEP 21 18:15:15.96	GET 200	wellbeing-intake.vercel...	/	
SEP 21 18:15:15.96	GET 200	wellbeing-intake.vercel...	/emergency	
SEP 21 18:10:37.22	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:10:38.210Z", "svc": "wellbein
SEP 21 18:08:45.88	GET 401	wellbeing-intake.vercel...	/api/v1/requests/me	{ "t": "2026-09-21T11:08:45.916Z", "svc": "wellbein
SEP 21 18:08:45.36	POST 405	wellbeing-intake.vercel...	/api/v1/services	
SEP 21 18:08:45.05	GET 200	wellbeing-intake.vercel...	/api/v1/health	{ "t": "2026-09-21T11:08:45.111Z", "svc": "wellbein
SEP 21 18:08:44.70	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:08:44.794Z", "svc": "wellbein
SEP 21 18:08:44.32	GET 200	wellbeing-intake.vercel...	/api/v1/health	{ "t": "2026-09-21T11:08:44.473Z", "svc": "wellbein
SEP 21 18:07:17.05	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:07:18.064Z", "svc": "wellbein
SEP 21 18:00:30.98	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T11:00:31.368Z", "svc": "wellbein
SEP 21 17:55:08.07	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T10:55:08.397Z", "svc": "wellbein
SEP 21 17:51:09.55	GET 200	wellbeing-intake.vercel...	/api/v1/services	{ "t": "2026-09-21T10:51:09.778Z", "svc": "wellbein
SEP 21 17:50:11.06	GET 200	wellbeing-intake.vercel...	/request/new	
SEP 21 17:50:10.70	GET 200	wellbeing-intake.vercel...	/request/new	
SEP 21 17:50:10.38	GET 200	wellbeing-intake.vercel...	/request/new	
SEP 21 17:50:10.38	GET 200	wellbeing-intake.vercel...	/request/new	

Logs 1 Total

Function

```
11:19:24.678 {
  t: "2026-09-21T11:19:24.677Z",
  svc: "wellbeing",
  cid: "team14-a5-c006d636-c04e-4d40-9653-2c1138a03ad4",
  method: "GET",
  path: "/api/v1/services",
  status: 200,
  role: "visitor",
  ms: 371
}
```

Request started SEP 21 18:19:23.39 GMT+7

Request ID jtsxv-1789989563395-840865d017e4

Path /api/v1/services

Host wellbeing-intake.vercel.app

Received in Singapore (sint)

Firewall Allowed

Function Invocation

Route /api/v1/services

Execution Duration / Maximum 868ms / 5m

External APIs GET jtwrgfcmfghfklddyatv.supabase.co...

Fluid 226 MB

Response finished in 1.3s

Deployment Information

Deployment ID dpl_8cBgkHfDt8bMS5ezAmH7zkCKs...

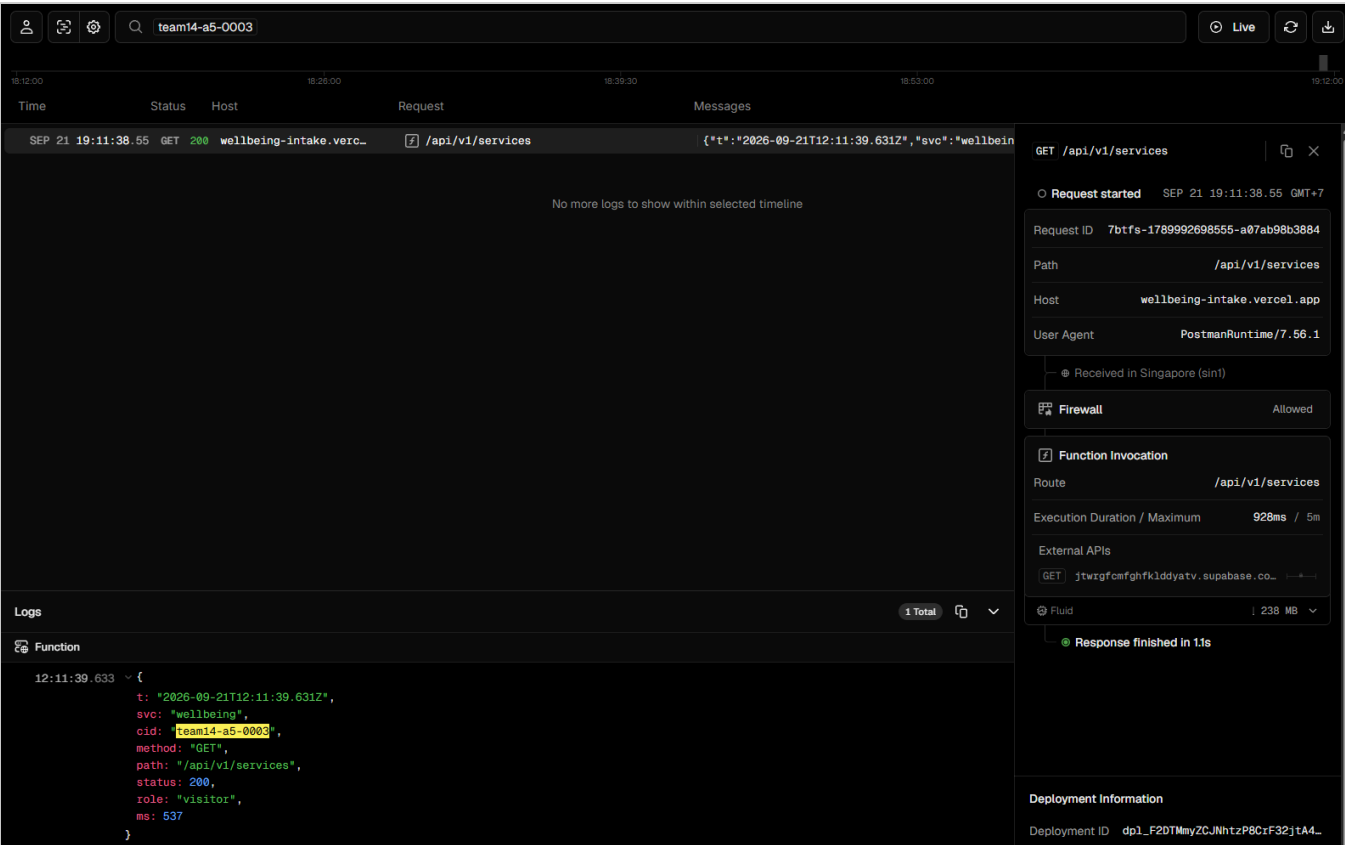
Checked from our side on 2026-09-21: GET https://helpdesk-api.team-helpdesk.workers.dev/wellbeing/services → 200, "degraded": false, all four services (counselling, health-clinic, physiotherapy, wellbeing-advising).

2c. Team 14's recovery call — correlation ID team14-a5-0003

In Team 14's test plan this label marks the first successful call to us after they simulated a Wellbeing outage on their own side (their Test 8). The outage itself never reaches us by design — they point their client at an unreachable host — so our log can only show the recovery:

```
{ "t": "2026-09-21T12:11:39.631Z", "svc": "wellbeing", "cid": "team14-a5-0003", "method": "GET", "path": "/api/v1/services", "status": 200, "role": "visitor", "ms": 537 }
```

19:11:38 UTC+7, User-Agent: PostmanRuntime/7.56.1, received in Singapore. The search for this label also returns one request only.



3. Webhook Receiver — we receive a partner's event

Implemented as FR-24 (PRD revision 3): POST /api/v1/webhooks/notification-hub receives Notification Hub delivery receipts (notification.delivered / notification.failed) carrying only our opaque reference; the signature is verified over the raw body and consumption is idempotent on eventId.

Field	Value
Sending partner	Self-run by Team 16, signed with our own HUB_INBOUND_SECRET (no Notification Hub partner — see Scope)
Our receiver URL	https://wellbeing-intake.vercel.app/api/v1/webhooks/notification-hub
Event type	notification.delivered
Received at	2026-09-21T00:56:20.136655Z (database received_at)
eventId	d277ab91-6e47-40d8-9c63-d65d8e46dc71

Incoming payload (exactly as received):

```
{"eventId":"d277ab91-6e47-40d8-9c63-d65d8e46dc71","type":"notification.delivered","occurredAt":"2026-09-21T00:56:16Z","source":"notification-hub","subject":"reference:demo","data":{"reference":"02193392-d6f3-48fc-af68-d887ddaa7349"}}}
```

Secret verification result — both outcomes:

Case	Signature header	Result	HTTP status	Correlation ID · response
Valid signature	sha256=a71e8c... (masked), x-signature-timestamp: 1789952176	verified	200	a5r-3-valid · { "received":true,"duplicate":false }
Tampered body — one character changed after signing (delivered → deliverex), same signature	sha256=a71e8c... (masked)	rejected	401	a5r-3-tampered · { "error":"invalid_signature" }

Server-side log lines (Vercel → Logs, production, region `sin1`) — our structured `kind:"webhook_in"` line for each delivery:

cid	Server time (UTC)	signature	status	duplicate	Work done
a5r-3-valid	2026-09-21T00:56:20.217Z	verified	200	false	one database write (External APIs: POST ...supabase.co)
a5r-3-duplicate	2026-09-21T00:56:20.453Z	verified	200	true	recognised as already seen
a5r-3-tampered	2026-09-21T00:56:20.658Z	rejected, reason mismatch	401	—	"No outgoing requests" — rejected in 9 ms, before touching the database

a5r-3-valid

07:29:0007:38:0007:42:4007:49:2007:59:00

TimeStatusHostRequestMessages

SEP 21 07:56:19.75 POST 200 wellbeing-intake.vercel... /api/v1/webhooks/notification-hub {"t":"2026-09-21T00:56:20.217Z","svc":"wellbein

No more logs to show within selected timeline

Logs1 Total

Function

```
00:56:20.217 {  
  t: "2026-09-21T00:56:20.217Z",  
  svc: "wellbeing",  
  kind: "webhook_in",  
  cid: "a5r-3-valid",  
  signature: "verified",  
  status: 200,  
  eventId: "d277ab91-6e47-40d8-9c63-d65d8e46dc71",  
  type: "notification.delivered",  
  duplicate: false  
}
```

Live

POST /api/v1/webhooks/notificati...

Request started SEP 21 07:56:19.75 GMT+7

Request ID nsxv2-1789952179754-5535abc5dbdf

Path /api/v1/webhooks/notification-hub

Host wellbeing-intake.vercel.app

User Agent curl/8.18.0

Received in Singapore (sin1)

Firewall Allowed

Function Invocation

Route /api/v1/webhooks/notification-hub

Execution Duration / Maximum 153ms / 5m

External APIs

POST jtwzqfcmghfklddytv.supabase.co...

Fluid 236 MB

Response finished in 465ms

Deployment Information

Deployment ID dp1_BHf6zpRANmFagLmFYyooVkoH4...

Environment production

a5r-3-duplicate

07:29:00

07:35:00

07:42:40

07:49:20

07:59:00

Time

Status

Host

Request

Messages

SEP 21 07:56:20.38 POST 200 wellbeing-intake.vercel_

/api/v1/webhooks/notification-hub

{ "t": "2026-09-21T00:56:20.453Z", "svc": "wellbeing"

No more logs to show within selected timeline

Logs

Function

00:56:20.454 {
 t: "2026-09-21T00:56:20.453Z",
 svc: "wellbeing",
 kind: "webhook_in",
 cid: a5r-3-duplicate,
 signature: verified,
 status: 200,
 eventId: "d277ab91-6e47-40d8-9c63-d65d8e46dc71",
 type: notification.delivered,
 duplicate: true
}

Request started SEP 21 07:56:20.38 GMT+7

Request ID jhtgt-1789952180387-28028f32dfa6

Path /api/v1/webhooks/notification-hub

Host wellbeing-intake.vercel.app

User Agent curl/8.18.0

Received in Singapore (sin1)

Firewall Allowed

Function Invocation

Route /api/v1/webhooks/notification-hub

Execution Duration / Maximum 53ms / 5m

External APIs

POST jtzwgcfmgfhklddyatv.supabase.co_

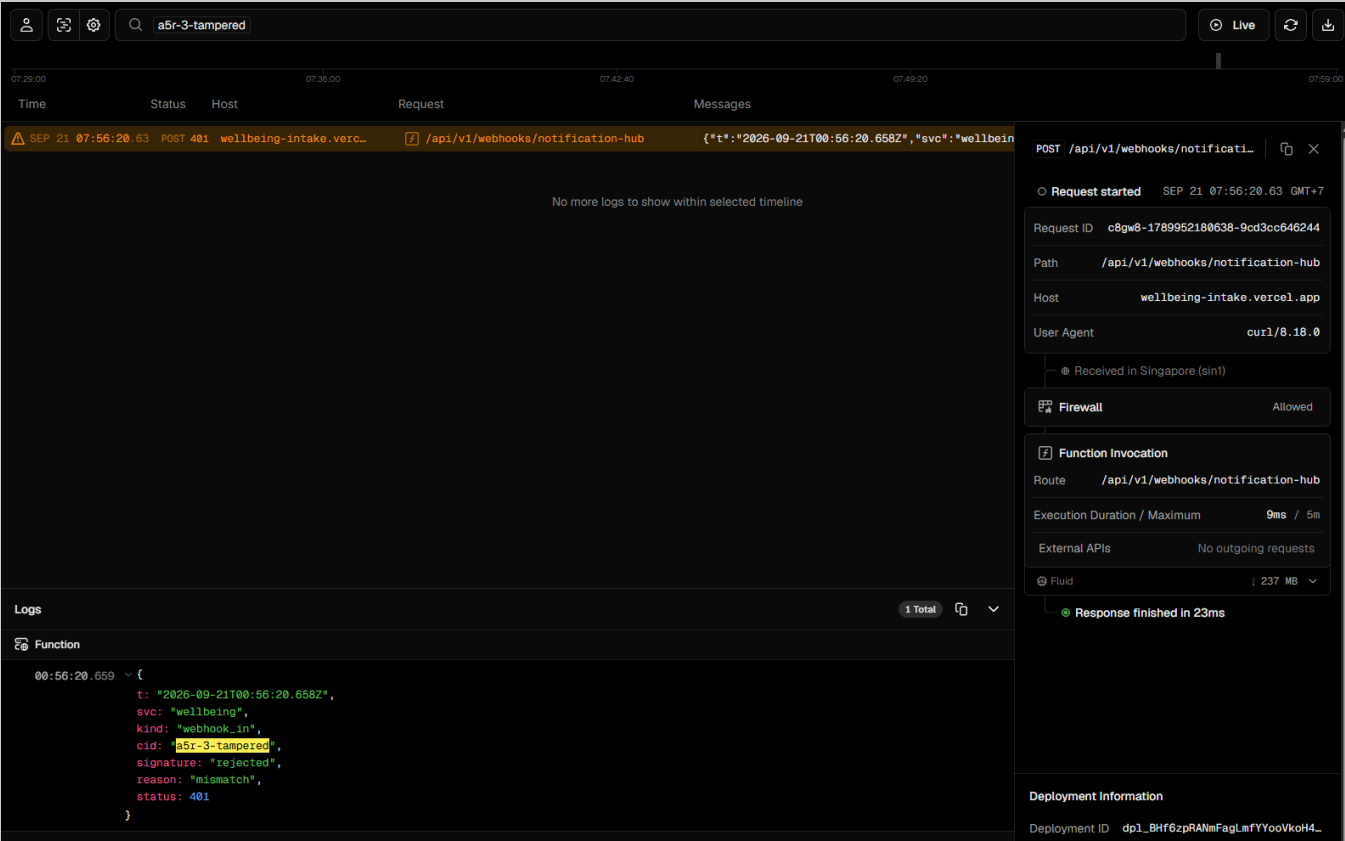
Fluid 237 MB

Response finished in 69ms

Deployment Information

Deployment ID dp1_BHff6zprANmfagLmFYyooVkoH4_

Environment production



Stored log — query output from the deployed database (`inbound_event` , filtered by that `eventId` — exactly one row):

```
[{"event_id": "d277ab91-6e47-40d8-9c63-d65d8e46dc71", "source": "notification-hub", "type": "notification.delivered", "reference": "02193392-d6f3-48fc-af68-d887ddaa7349", "received_at": "2026-09-21T00:56:20.136655+00:00"}]
```

4. Webhook Sender — we send an event to a partner

What this proves: FR-18 / NFR-17 — a signed `appointment.reminder` reaches the Notification Hub target in the platform envelope (here: the contract mock hub).

Internal trigger action: student books an appointment (`POST /api/v1/appointments`) → reminder reaches its lead time → dispatcher runs (`POST /api/v1/internal/events/dispatch`).

Step	Timestamp	Correlation ID	Evidence
Receiver in this capture: our contract mock hub (<code>HUB_WEBHOOK_URL=https://wellbeing-intake.vercel.app/api/v1/mock/hub</code>), which checks the HMAC signature, the platform envelope and the <code>data</code> allowlist before answering <code>202</code> . There is no Notification Hub partner — see Scope.			

Step	Timestamp (UTC)	Correlation ID	Evidence
Booking created	2026-09-21T00:56:24Z	a5r-4-booking	201 { "id": "65132edf-0d2d-4882-9a4e-c340fec2f2b5", "startAt": "2026-09-21T06:56:19.269+00:00", "status": "confirmed" }
Outbox row inserted (K-18)	same transaction window	—	event_id 6e6f17f3-9038-449b-b757-9220dfe9ef77, type appointment.reminder, reference = the appointment id
Dispatcher delivered (inline, right after booking)	2026-09-21T00:56:24.113613Z	a5r-4-booking	attempts 1, last_status 202; log screenshots below

Outgoing payload — data has exactly 3 fields; no service, practitioner, reason or triage level. The delivered row is nulled (K-19), so this is the data block of the identically-built reminder in §6a, read from the outbox while it was still pending:

```
{ "message": "You have an appointment.", "reference": "fcbe8a2f-0847-4ac3-8a67-1a6f97d825c1",
  "appointmentAt": "2026-09-21T08:01:25Z" }
```

with subject student:3d77124f-5e2c-4c35-b7da-b8e74d1d0d06 (the recipient's opaque identity reference, a seeded demo user).

Signature header sent: X-Signature: sha256=<hex> with X-Signature-Timestamp — the mock hub recomputed the HMAC over "<timestamp>.<raw body>" and accepted it (202). The header value itself is not logged by design, so it is not reproduced here.

Partner response log: HTTP status 202 from the mock hub (it answers 401 on a bad signature, 400 on a bad envelope, 422 on data outside the contract).

Server-side log lines — both ends of the delivery, same eventId 6e6f17f3-9038-449b-b757-9220dfe9ef77 (Vercel → Logs, search a5r-4-booking; the correlation ID is propagated from the booking request onto the outbound webhook):

Side	Request	Server time (UTC)	Log line
Sender	POST /api/v1/appointments → 201	2026-09-21T00:56:24.170Z	kind:"webhook", type:"appointment.reminder", attempt:1, status:202, outcome:"delivered"; External APIs shows the outbound POST wellbeing-intake.vercel.app/api/v1/mock/hub
Receiver	POST /api/v1/mock/hub → 202, user agent node	2026-09-21T00:56:24.139Z	svc:"mock-hub", same eventId, accepted:true — logged only after the HMAC signature, the envelope and the data allowlist all passed

07:29:00
07:35:00
07:42:40
07:49:20
07:59:05

Time	Status	Host	Request	Messages
SEP 21 07:56:24.12	POST 202	wellbeing-intake.vercel	/api/v1/mock/hub	{ "t": "2026-09-21T00:56:24.139Z", "svc": "mock-hub" }
SEP 21 07:56:23.90	POST 201	wellbeing-intake.vercel	/api/v1/appointments	{ "t": "2026-09-21T00:56:24.171Z", "svc": "wellbeing-intake.vercel" }

No more logs to show within selected timeline

Logs

Function

```

00:56:24.173 {
  t: "2026-09-21T00:56:24.170Z",
  svc: "wellbeing",
  kind: "webhook",
  cid: "a51-4-booking",
  eventId: "6e6f17f3-9038-449b-b757-9220dfe9ef77",
  type: "appointment.reminder",
  attempt: 1,
  status: 202,
  outcome: "delivered"
}

00:56:24.173 {
  t: "2026-09-21T00:56:24.171Z",
  svc: "wellbeing",
  cid: "a51-4-booking",
  method: "POST",
  path: "/api/v1/appointments",
  status: 201,
  role: "student",
  ms: 242
}
          
```

2 Total

Request started

Request ID 417kq-1789952183906-9ab380dfe...

Path /api/v1/appointments

Host wellbeing-intake.vercel.app

User Agent curl/8.18.0

Received in Singapore (sin1)

Firewall Allowed

Function Invocation

Route /api/v1/appointments

Execution Duration / Maximum 251ms / 5m

External APIs

GET jtwgfcmgfghfklddyatv.supabase...

GET jtwgfcmgfghfklddyatv.supabase...

POST jtwgfcmgfghfklddyatv.supabase...

POST jtwgfcmgfghfklddyatv.supabase...

POST jtwgfcmgfghfklddyatv.supabase...

POST wellbeing-intake.vercel.app/ap...

POST jtwgfcmgfghfklddyatv.supabase...

Fluid 1.51 MB

Response finished in 268ms

Deployment Information

Deployment ID dp1_Bhf6zprANmFagLmfYyooVko...

SEP 21 07:56:23.90 GMT+7

[illegible]

After delivery — outbox row, delivered at set and subject / payload nulled (K-19):

```
[{"event_id":"6e6f17f3-9038-449b-b757-9220dfe9ef77","type":"appointment.reminder","reference":"65132edf-0d2d-4882-9a4e-c340fec2f2b5","attempts":1,"last_status":202,"next_attempt_at":"2026-09-21T00:56:45.051186+00:00","delivered_at":"2026-09-21T00:56:24.113613+00:00","failed_at":null,"subject":null,"payload":null}]
```

5. Idempotency Proof — same request twice, one record

Two candidates; submit at least one, ideally both.

5a. POST /api/v1/requests with the same submission_key (K-17)

	Req 1	Req 2 (replay)
Timestamp (UTC)	2026-09-20T22:33:09Z	2026-09-20T22:33:10Z
Correlation ID	a5-5a-req1	a5-5a-req2
submission_key	19267117-022d-4128-a196-1348548429b3	same
HTTP status	201 Created	200 OK — returns the existing request
Returned request id	a9e01cc4-fac0-4851-9336-3007f9d0380d	same id
created flag	true	false

Payload — byte-identical for both requests (obviously fake description, as required):

```
{"serviceId":"689dfeea-2a43-4aa9-87b4-55cfacaf29bc","structuredDescription":"demo-sentinel-A5","preferredTimes":"any","triageLevelId":1,"submissionKey":"19267117-022d-4128-a196-1348548429b3"}
```

Response 1:

```
{"id":"a9e01cc4-fac0-4851-9336-3007f9d0380d","status":"submitted","triageLevelId":1,"submittedAt":"2026-09-20T22:33:13.184206+00:00","created":true,"acute":false}
```

Response 2 — same id, same submittedAt, created:false:

```
{"id":"a9e01cc4-fac0-4851-9336-3007f9d0380d","status":"submitted","triageLevelId":1,"submittedAt":"2026-09-20T22:33:13.184206+00:00","created":false,"acute":false}
```

DB proof of single creation:

```
select count(*) from request where submission_key = '19267117-022d-4128-a196-1348548429b3';
-- expected: 1
```

Result from the deployed database (exact count via the REST API, Prefer: count=exact): Content-Range: 0-0/1 → **1 row**. (To reproduce: run the SQL above in Supabase → SQL Editor.)

5b. Inbound webhook delivered twice with the same `eventId` (from §3)

	Delivery 1	Delivery 2 (duplicate)
Timestamp (UTC, server)	2026-09-21T00:56:20Z	2026-09-21T00:56:20Z
Correlation ID	a5r-3-valid	a5r-3-duplicate
<code>eventId</code>	d277ab91-6e47-40d8-9c63-d65d8e46dc71	same (same body, same signature)
HTTP status	200	200 — acknowledged, not reprocessed
Response	{"received":true,"duplicate":false}	{"received":true,"duplicate":true}

DB proof — one stored row for that `eventId` (the primary key on `inbound_event.event_id` makes a second insert impossible):

```
[{"event_id":"d277ab91-6e47-40d8-9c63-d65d8e46dc71","source":"notification-hub","type":"notification.delivered","reference":"02193392-d6f3-48fc-af68-d887ddaa7349","received_at":"2026-09-21T00:56:20.136655+00:00"}]
```

6. Degradation Proof — a dependency breaks, we keep working, we recover automatically

6a. Notification Hub unreachable (primary — partner dependency)

How it was broken: the hub target was switched into outage mode — `PUT /api/v1/mock/hub {"fail":true}` with the machine credential — so every delivery received `503 {"error":"mock_hub_outage"}` over real HTTP.

One event throughout: `event_id 7639a8f8-ba9a-426a-b960-ab9be25715b0`.

Moment	Timestamp (UTC)	Evidence
Breakage	2026-09-20T22:51:05Z	cid a5-6a-break → 200 {"fail":true}
Booking during outage still succeeds	2026-09-20T22:51:06Z	cid a5-6a-booking → 201, fallback JSON below
Attempt 1 fails (inline)	22:51:06Z	outbox: attempts 1, last_status 503, next_attempt_at 22:51:39Z (≈ 30 s later), delivered_at null
Attempt 2 fails (dispatcher)	2026-09-20T22:51:49Z	cid a5-6a-retry2 → {"enqueued":0,"claimed":1,"delivered":0,"retrying":1,"parked":0}; outbox: attempts 2, last_status 503, next_attempt_at 22:53:03Z (≈ 70 s later — interval growing, same event_id)
Dependency restored	2026-09-20T22:52:17Z	cid a5-6a-restore → {"fail":false}

Moment	Timestamp (UTC)	Evidence
Automatic recovery	2026-09-20T22:53:07Z	cid a5-6a-recover → { "enqueued":0, "claimed":1, "delivered":1, "retrying":0, "parked":0 }; outbox: attempts 3, last_status 202, delivered_at 22:53:21Z, subject and payload nulled

What "automatic" means here: no row was edited and nothing was re-queued by hand. Recovery was the ordinary dispatcher call picking up the row when its `next_attempt_at` came due. In production that call is made by the scheduled GitHub Actions workflow (`dispatch.yml`, every 3 hours); for this capture we invoked the same endpoint at the due times instead of waiting for the schedule.

Pending outbox row during the outage (it still holds its payload — exactly the three allowed fields):

```
[{"event_id":"7639a8f8-ba9a-426a-b960-ab9be25715b0","type":"appointment.reminder","attempts":1,"last_status":503,"next_attempt_at":"2026-09-20T22:51:39.850353+00:00","delivered_at":null,"failed_at":null,"subject":"student:3d77124f-5e2c-4c35-b7da-b8e74d1d0d06","payload":{"message":"You have an appointment.", "reference":"fcbe8a2f-0847-4ac3-8a67-1a6f97d825c1", "appointmentAt":"2026-09-21T08:01:25Z"}}]
```

The same row after recovery:

```
[{"event_id":"7639a8f8-ba9a-426a-b960-ab9be25715b0","attempts":3,"last_status":202,"delivered_at":"2026-09-20T22:53:21.05775+00:00","failed_at":null,"subject":null,"payload":null}]
```

The appointment was never affected by the outage:

```
[{"id":"fcbe8a2f-0847-4ac3-8a67-1a6f97d825c1","status":"confirmed","reminder_published_at":"2026-09-20T22:51:09.759946+00:00"}]
```

Fallback JSON output (the booking response returned to the student while the Hub was down — identical in shape to a normal booking; the student is never told, and never blocked, by a notification outage):

```
{"id":"fcbe8a2f-0847-4ac3-8a67-1a6f97d825c1","startAt":"2026-09-21T08:01:25.449+00:00","status":"confirmed"}
```

6b. LLM unavailable (AI capability with deterministic fallback — FR-23, BR-24)

Dependency: Groq (`openai/gpt-oss-20b`), the LLM behind the "help me choose" helper. Deploy evidenced here: `73b18dc` .

How it was broken: Groq's free tier allows 8,000 tokens per minute. We used that allowance up with one oversized request sent **from outside the app** with the same API key, so Groq answered every further request — including our deployed app's — with `429 rate_limit_exceeded` (`retry-after: 55`). This is a real provider-side failure, not a flag flipped in our own code.

Interface showing logs for deployment **a5-6b-during**. The timeline shows two POST requests to `/api/v1/finder/suggest` from `wellbeing-intake.vercel.app` at 07:33:36.21 and 07:33:35.77. The right sidebar details the first request, showing it was received in Singapore, passed the firewall, and the function invocation completed in 134ms. The function logs show a fallback response due to an unavailable AI service.

```

{
  t: "2026-09-21T00:33:36.381Z",
  svc: "wellbeing",
  kind: "finder",
  cid: "a5-6b-during2",
  mode: "fallback",
  reason: "ai_unavailable",
  matches: 1,
  ms: 127
}

```

Interface showing logs for deployment **a5-6b-recovered**. The timeline shows a POST request to `/api/v1/finder/suggest` at 07:34:44.55. The right sidebar details the request, showing it was received in Singapore, passed the firewall, and the function invocation completed in 926ms. The function logs show a successful AI response.

```

{
  t: "2026-09-21T00:34:45.502Z",
  svc: "wellbeing",
  kind: "finder",
  cid: "a5-6b-recovered",
  mode: "ai",
  reason: null,
  matches: 2,
  ms: 920
}

```

Request body in every call: `{"text": "trouble sleeping before exams"}`.

Fallback JSON output during the outage — a ranked list of seeded service IDs from the deterministic keyword matcher; the visitor gets an answer, not an error:

```
{"mode":"fallback","serviceIds":["6c940476-f362-4090-b96a-40d75ed2564b"]}}
```

Same call after recovery — the model ranks a second relevant service the keyword matcher did not find:

```
{"mode":"ai","serviceIds":["6c940476-f362-4090-b96a-40d75ed2564b","689dfeea-2a43-4aa9-87b4-55cfacaf29bc"]}}
```

In both modes the response carries **only seeded service IDs** — model-written text never reaches a screen (BR-24).

Also captured earlier (2026-09-20T23:09:24Z, cid a5-6b-fallback, deploy 818596e): with the capability switched off entirely (AI_FINDER_ENABLED=false, no key) the same call returned the same mode:"fallback" answer.

Automated coverage of the fallback triggers: tests/unit/finder-ai.test.ts stubs the network and asserts a fallback on HTTP 500, HTTP 429, a timeout, a network failure, free text instead of JSON, JSON of the wrong shape, an id outside the catalogue, and more than three ids; tests/integration/platform.test.ts covers AI switched off, no key, and daily budget exhausted. All green in CI.

Sign-off

Partner team	Contact	What they confirmed	Date
Team 14 — Helpdesk	Pupattarak Masomjit (6731503115)	\$2	2026-09-21

Appendix — how to capture each proof with this codebase

Set once (Git Bash). Use the deployed URL for the real submission; http://localhost:3000 for a dry run.

```
HOST=https://wellbeing-intake.vercel.app
CID=a5-$(date +%Y%m%dT%H%M%S)          # one correlation id per proof ties screenshots to log lines
login() { curl -s -X POST $HOST/api/v1/session -H 'content-type: application/json' -H "origin: $HOST" \
  -d '{"email":"'${1}'","password":"'demo-password-16'" }' | node -pe
  'JSON.parse(require("fs").readFileSync(0)).accessToken'; }
STUDENT=$(login student.a@demo.test); COUNSELLOR=$(login counsellor.1@demo.test)
```

Server log lines are single-line JSON containing "cid":"<your CID>" — in Vercel: Project → Logs → search the CID.

1 • Consumer (Team 01). Set IDENTITY_MODE=campus, IDENTITY_JWKS_URL, IDENTITY_ISSUER. Obtain a token from Team 01, then curl -i \$HOST/api/v1/me -H "Authorization: Bearer \$TEAM01_TOKEN" -H "X-Correlation-Id: \$CID". A 200 with your role proves the adapter fetched Team 01's key set and verified the token. Screenshot the response and the partner URL (IDENTITY_JWKS_URL) being fetched; repeat with a tampered token to show 401.

2 • Provider. Ask the partner to run (and screenshot) curl -i \$HOST/api/v1/services -H "X-Correlation-Id: team13-a5-0001". Your internal log is the line with that cid.

3 • Webhook receiver. Partner (or a dry run with their secret) POSTs a signed receipt:

```
BODY='{ "eventId":"'$(node -p
'crypto.randomUUID()')'", "type": "notification.delivered", "occurredAt":"'$(date -u +%Y-%m-
%dT%H:%M:%SZ)'" , "source": "notification-hub", "subject": "reference:demo", "data": { "reference": "'$(node -p
'crypto.randomUUID()')'" } }'
TS=$(date +%s)
SIG=sha256=$(printf '%s.%s' "$TS" "$BODY" | openssl dgst -sha256 -hmac "$HUB_INBOUND_SECRET" | awk
'{print $NF}')
curl -i -X POST $HOST/api/v1/webhooks/notification-hub -H 'content-type: application/json' \
-H "x-signature: $SIG" -H "x-signature-timestamp: $TS" -H "X-Correlation-Id: $CID" -d "$BODY"
```

Valid → 200 {"received":true,"duplicate":false} and log "signature":"verified". Change one character of \$BODY after signing → 401 and log "signature":"rejected","reason":"mismatch". Stored log: select * from inbound_event order by received_at desc limit 5;

4 • Webhook sender. Book a slot that starts within 24 h (the seed creates one per practitioner ~20 h ahead): submit a request as the student, open it as the counsellor, book it as the student (UI or API). The booking triggers an inline dispatch. Evidence: the kind:"webhook" log line (eventId, attempt, status, outcome:"delivered"), the payload as Team 20 received it, their confirmation, and select event_id,type,attempts,last_status,delivered_at,subject,payload from outbound_event order by occurred_at desc limit 3; (subject and payload are null after delivery — K-19). To show the payload *before* delivery, unset HUB_WEBHOOK_URL, book, query the row, then restore the URL and run the dispatcher.

5 • Idempotency. 5a: send the *same* body twice —

```
KEY=$(node -p 'crypto.randomUUID()'); SVC=$(curl -s $HOST/api/v1/services | node -pe
'JSON.parse(require("fs").readFileSync(0)).services[0].id')
REQ='{ "serviceId":"'${SVC}', "structuredDescription": "demo-sentinel-
A5", "preferredTimes": "any", "triageLevelId": 1, "submissionKey": "'$KEY'" }'
curl -i -X POST $HOST/api/v1/requests -H "Authorization: Bearer $STUDENT" -H 'content-type:
application/json' -d "$REQ" # 201 created:true
curl -i -X POST $HOST/api/v1/requests -H "Authorization: Bearer $STUDENT" -H 'content-type:
application/json' -d "$REQ" # 200 created:false, same id
```

DB proof: select count(*) from request where submission_key = '<KEY>'; → 1. 5b: run the proof-3 curl twice with the same \$BODY → second response "duplicate":true; select count(*) from inbound_event where event_id = '<eventId>'; → 1.

6 • Degradation. 6a (Hub down): curl -X PUT \$HOST/api/v1/mock/hub -H "Authorization: Bearer \$DISPATCH_TOKEN" -H 'content-type: application/json' -d '{"fail":true}' (or have Team 20 take their endpoint down at an agreed time) → book a near-term slot: the booking still returns 201 (the fallback JSON) → logs show outcome:"retry_scheduled" with a growing nextAttemptAt, same eventId → restore with {"fail":false} → the next scheduled dispatch (or POST /internal/events/dispatch) logs outcome:"delivered" with no manual data fix: that is the automatic-recovery log. 6b (LLM down): with AI_FINDER_ENABLED=true, break LLM_API_KEY → curl -s -X POST \$HOST/api/v1/finder/suggest -H 'content-type: application/json' -d '{"text":"trouble sleeping"}' → {"mode":"fallback",...}; restore the key → {"mode":"ai",...}. The kind:"finder" log lines give the breakage and recovery timestamps.